

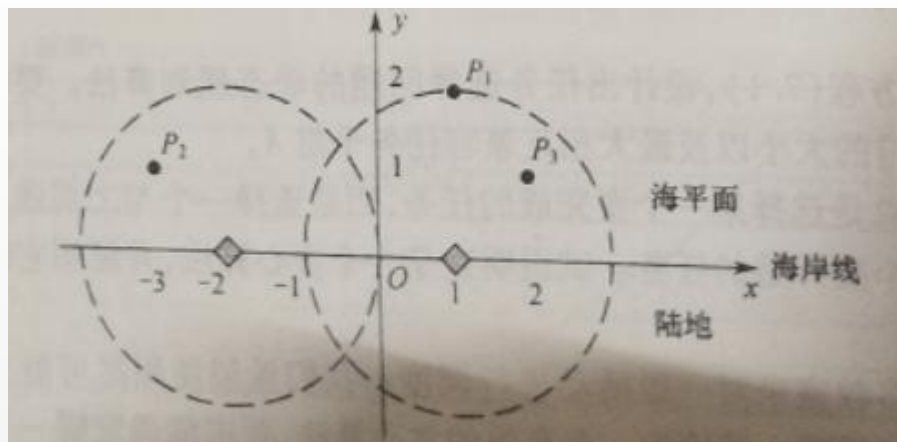
第六周作业

1.证明背包问题具有最优子结构性质

定理 6.4 假设子问题 (i, w) 的最优装载中含有物品 i , 则其中子问题 $(i-1, w-w_i)$ 的装载方案也一定是最优的。

证明: 可用反证法证明。假设子问题 $(i-1, w-w_i)$ 的装载方案 p 不是最优的, 则有一个更优的装载方案 p' , 将 p' 替换 p 然后再加上物品 i 将会比原来的最优装载的价值更大, 与假设矛盾。

2.假设海岸线为一条直线，陆地和海平面一边有很多小岛，每个小岛可视为一个点。海岸上可布控多个雷达，每个雷达监测半径为 d ，若小岛到雷达的范围不超过 d ，那么小岛就在雷达可控范围内，如附件所示， x 轴为海岸，海位于 x 轴上方，小岛坐标为 (x, y) 。给定一些小岛的坐标和雷达控制范围 d ，设计算法计算控制所有小岛所需的雷达数目。



该题为poj1328。

可以使用贪心的想法，从左到右建立雷达，要尽量多地覆盖岛屿。

以岛屿为圆心，以 d 为半径画圆，如果画出的圆与轴没有交点，则不能实现。存在交点的话计算出第 i 个岛屿与 x 轴的交点坐标保存在结构体数组 $rad[i].start$ 与 $rad[i].end$ 中。

对结构体数组排序，按照 $rad[i].end$ 进行升序排列。

然后一次从左到右找雷达。对于 $rad[i].end$ 为当前最右边的左坐标，对于下一个岛屿，如果 $rad[i].start < rad[i].end$ ，则不需要建立新的雷达，需要更新下一个岛屿。否则需要建立新的雷达。

3.如何修改背包问题的动态规划算法,使得它只需要 $\Theta(C)$ 空间,其中C为背包容量

由 0-1 背包的状态转移方程 $dp[i][j] = \max\{dp[i-1][j-v[i]]+w[i], dp[i-1][j]\}$ 的特点, 知道当前状态仅依赖前一状态的剩余容量与当前物品重量 $w[i]$ 的关系。根据这个特点, 我们可以将 dp 降到一维即 $dp[j] = \max\{dp[j], dp[j-v[i]]+w[i]\}$ 。从这个方程中我们可以发现, 有两个 $dp[j]$, 但是要区分开。等号左边的 $dp[j]$ 是当前 i 的状态, 右边括号中的 $dp[j]$ 是第 $i-1$ 状态下的值。

```
#include<bits/stdc++.h>
using namespace std;
int bag,n,v[101],w[101],dp[1001];
int main()
{
    scanf("%d%d",&bag,&n);
    for(int i=1;i<=n;i++)scanf("%d%d",&v[i],&w[i]);
    for(int i=1;i<=n;i++)
    {
        for(int j=bag;j>=v[i];j--)
        {
            dp[j]=max(dp[j],dp[j-v[i]]+w[i]);
        }
    }
    cout<<dp[bag]<<endl;
    return 0;
}
```

4.定义0 / 1 / 2背包问题为: $\max\{\sum v_i x_i\}$, 约束条件为: $\sum w_i x_i \leq c$ 且 $x_i \in \{0, 1, 2\}$, $1 \leq i \leq n$ 。设P的定义同0 / 1背包问题中的定义。

1) 从0 / 1 / 2背包问题中推出类似于0/1背包的递归公式。

2) 假设w为整数。编写一个类似于0/1背包的程序来计算二维数组P, 然后确定最优分配的x 值。

3) 分析上述算法的时间复杂度。

$p[i, w]$: 当背包容量为 w 时, 可以从物品 1 到 i 获得的最大价值

(1) 不装入物品 i $p[i, w] = p[i-1, w]$

(2) 装入 1 个物品 i $p[i, w] = p[i-1, w-w_i] + v_i$

(3) 装入 2 个物品 i $p[i, w] = p[i-1, w-2w_i] + 2v_i$

递归方程为:

$p[i, w] = \max\{p[i-1, w], p[i-1, w-w_i] + v_i, p[i-1, w-2w_i] + 2v_i\}$

012knapsack(W, v, w)

1 **for** $i \leftarrow 2$ to n **do**

2 **for** $w \leftarrow 0$ to W **do**

3 **if** $2w_i \leq w$ **then**

4 $p[i, w] \leftarrow \max\{p[i-1, w], p[i-1, w-w_i] + v_i, p[i-1, w-2w_i] + 2v_i\}$

5 **else if** $w_i \leq w$ **then**

6 $p[i, w] \leftarrow \max\{p[i-1, w], p[i-1, w-w_i] + v_i\}$

7 **else**

8 $p[i, w] \leftarrow p[i-1, w]$

时间复杂度为 $O(NC)$

第七周作业

1.假设你有一个数组，其中第 i 个元素是第 i 天给定股票的价格。设计算法以找到最大利润。你可以根据需要完成尽可能多的交易（即，多次买入并卖出一股股票）。注意：您不能同时进行多笔交易（即，您必须在再次购买之前卖出股票）

例如：给定数组[7,1,5,3,6,4]，算法输出：7

说明：第二天买入（价格：1）第三天卖出（价格：5）利润： $5-1=4$ ，然后第四天买入（价格：3）并在第五天卖出（价格：6），利润 $6-3=3$ ，总利润： $4+3=7$

要求给出测试案例，设计算法，并给出算法在你的测试案例上计算得到的结果。

```
1. maxpro ( A, n)
   if n==1
       return 0
   ret ← 0
   for i ← 1 to n do:
       if (A[i] > A[i-1])
           ret += A[i] - A[i-1]

   return ret
```

测试案例：[7 4 3 5 8 7 9] 算法输出：7

说明：要前一天小于当天价格才进行买卖

则 $ans = 5-3 + 8-5 + 9-7 = 7$

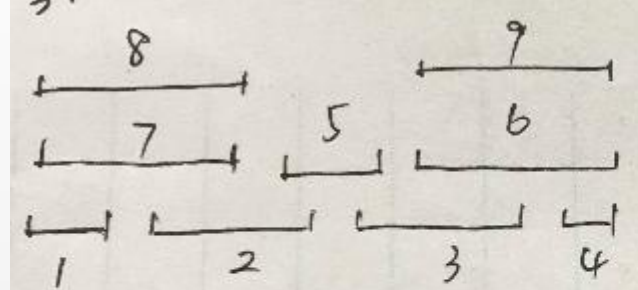
Answer from 何依娜

2.根据任务选择问题的递推表达式，设计动态规划算法。

```
DPtaskSelect()
1  sort by finish time
2  for i ← 0 to n+1 do
3    for j ← 0 to i-1 do
4      c[i,j] ← 0
5  for i ← 0 to n+1 do
6    for j ← i+1 to n+1 do
7      max1 ← 0
8      for k ← i+1 to j-1 do
9        if task[k].s ≥ task[i].f and task[k].f ≤ act[j].s then
10          if c[i,k]+c[k,j]+1 > max1 then
11            max1 ← c[i,k]+c[k,j]+1
12          c[i,j] ← max1
13  return c[0,n+1]
```


在活动安排问题中,还可以有其他的贪心选择方案,但并不能保证产生最优解。给出一个例子,说明若选择具有最短时段的相容活动作为贪心选择,得不到最优解;若选择覆盖未选择活动最少的相容活动作为贪心选择,也得不到最优解。

3.



可知,最优解为 a_1, a_2, a_3, a_4 .

(1) 若选具有最短时段的相容活动作为贪心选择,则会得到: a_1, a_5, a_4 , 显然不是最优解。

(2) 若选覆盖活动最少的相容活动作为贪心选择,则会选择 a_5 — 一旦有了 a_5 则最多只能选3个活动,如 a_1, a_5, a_4 或 a_5, a_6, a_7 / a_5, a_8, a_9 , 则显然也不是最优解。

Answer from 詹翁怡

若在 0-1 背包问题中,各物品依重量递增排列时,其价值恰好依递减序排列。对这个特殊的 0-1 背包问题,设计一个有效算法找出最优解,并说明算法的正确性。

4. 输入: 物品重量 $w[i]$, 物品价值 $v[i]$
输出: 背包最多可放重量 W
输出: 背包中可放的最大价值

(1) 对物品按重量递增排序
(2) 判断剩余重量 res (初值为 W) 与 $w[i]$ 的大小 (i 初值为 1, 表示排序后的第一件物品), 如果 $res > w[i]$, 执行 (3), 否则 $i++$, 再判断, 如果 $i > n$, 执行 (4), 否则执行 (2)
(3) 将 i 物品装入背包, $i++$, $res = w[i]$, $max_value += v[i]$, 判断 res 是否大于 0, 如果为 0, 执行 (4), 否则, 执行 (2)
(4) 输出 max_value

```
def M-value(w, v, W):  
    res = W  
    ans = 0  
    for i in range(1, n+1):  
        if res < w[i]:  
            continue  
        else:  
            ans += v[i]  
            res -= w[i]  
    return ans
```

贪心选择性质的证明
设最优解中背包所含物品为 $i, \dots, j$, 那么 i 一定是排序后最前面的物品且满足 $w[i] \leq res$
反证: 如果存在物品 k ($1 \leq k < i$), 那么由于物品按重量递增排序, 故 $w[k] < w[i]$ 且 $v[k] > v[i]$
则我们可以把 i 替换为 k , 此时总价值为原价值 + $v[k] - v[i] >$ 原价值, 与假设矛盾, 故结论成立

最优子结构的证明
设在背包最大重量为 W_i 时的最优解物品为 $i, \dots, k_j$, 那么一定有背包最大重量在 $(W - w_j)$ 时最优解为 $i, \dots, k$
反证: 假设 $i, \dots, k$ 不是背包最大重量在 $W - w_j$ 时的最优解, 那么我可以用最优解替换它, 再用最优解加上 j , 此时得到的解应该是在 W_i 时比假设更优的解, 与假设矛盾, 故结论成立

Answer from 沈欣海

第八周作业

1. 证明单源最短路径问题的贪心选择性质

1. 证明: 先证最优子结构性质,

即证若 $v, v_1, v_2, \dots, v_i, u$ 为最短路径, 那么 $v, v_1, v_2, \dots, v_i, v_i$ 为 v 到 v_i 的最短路径

证明: 设有另外一条路径比 $v, v_1, v_2, \dots, v_i$ 更短, 则记这条

路径长度为 d_2 , 那么有 $d_1 > d_2$ ①

但对 v 到 u 的过程, 有 $\text{dist}[u] = d_1 + \text{ars}[v_i][u] < d_2 + \text{ars}[v_i][u]$
即 $d_1 < d_2$ ②

①与②矛盾, 最优子结构证毕.

贪心选择性质.

证明: (1) 当 $|S|=1$ 时, 对所有的 u 属于 S , 都有 $\text{dist}[u] = \text{ars}[v][u]$
此时, $\text{dist}[u]$ 显然都为最小值成立

(2) 假设, 当 $|S|=k$ 时成立,

那么当 $|S|=k+1$ 时, 按贪心规则, 路径上又经过 S 中的点,

从 $V-S$ 中选择一点 x , 并使 $\text{dist}[u]$ 是 $\text{dist}[x]$, $x \in V-S$ 中最小的。

假设最短路径还经过了除了 u 之外别的 $V-S$ 中的点, 记该点为 x ,

那么则有 $\text{dist}[x] + \text{ars}[x][u] < \text{dist}[u]$

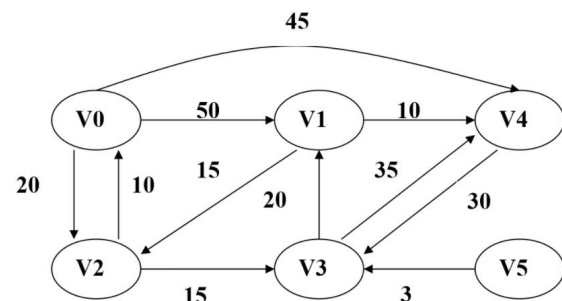
这与 u 的选取方式相矛盾,

∴ 贪心选择可以保证新纳入的点的路径为最佳。

即 $|S|=k+1$ 时成立

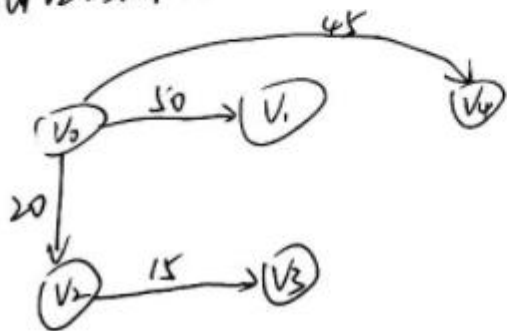
(3) 由归纳步骤与基础, 贪心选择性质证毕。

● 求v0出发的 最短路径



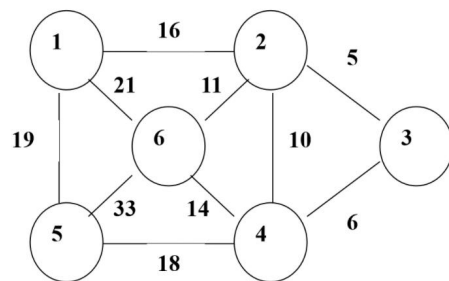
2. 迭代

	S	u	dist[1]	dist[2]	dist[3]	dist[4]	dist[5]
初始	$\{V_0\}$	-	50	20	∞	45	∞
1	$\{V_0, V_2\}$	V_2	50	20	35	45	∞
2	$\{V_0, V_2, V_3\}$	V_3	50	20	35	45	∞
3	$\{V_0, V_2, V_3, V_4\}$	V_4	50	20	35	45	∞
4	$\{V_0, V_2, V_3, V_4\}$	V_1	50	20	35	45	∞

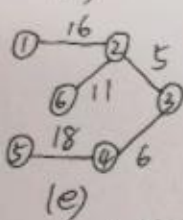
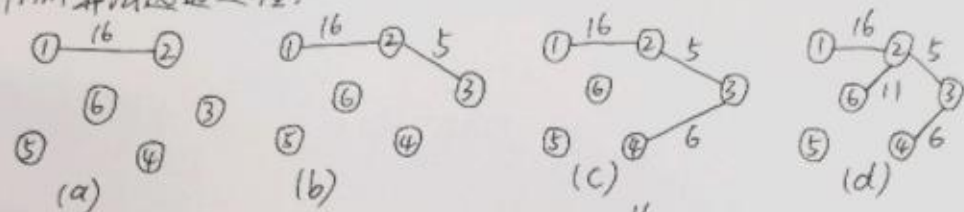


Answer from 魏智杰

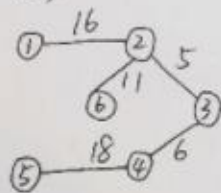
求最小生成树



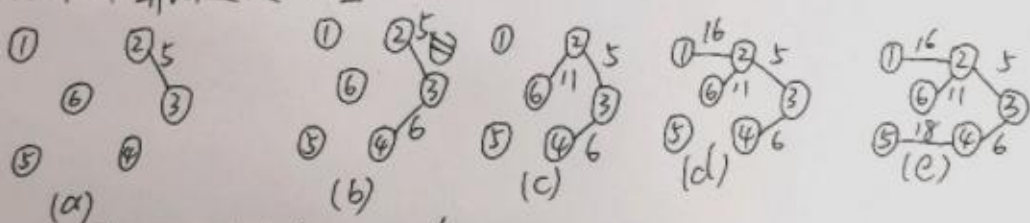
3. Prim算法选边过程:



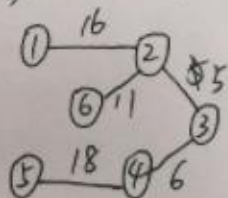
所以最小生成树为



Kruskal算法选边过程:



所以最小生成树为



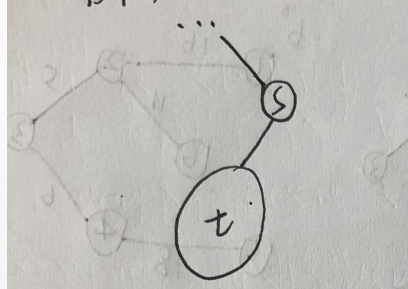
Answer from 黄伟中

4. 证明：一颗不满的二叉树不可能与一种最优前缀编码对应。

4.

对于一棵不满的二叉树 T , 其必然存在内部结点 s , s 只有一个子节点

例如:



t 可能只有一个结点也可能是树

设 s 的深度为 l_1 , t 的根结点深度为 l_2

我们用 t 的根结点取代 s 得到树 T'

那么:

$$B(T') \leq B(T) - 1$$

($B(T) - 1$ 是指 t 中只有一个结点时, 用该点替代 s 得到新树总编码长度会减少 1)

因此, 不满二叉树不可能与一种最优前缀编码对应。

Answer from 沈欣海

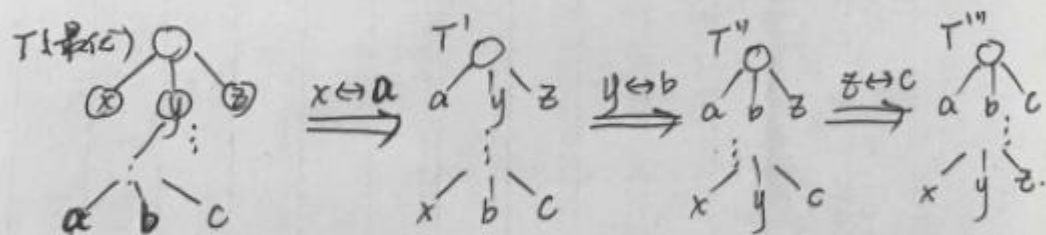
5. 将赫夫曼编码推广至三进制编码, 并证明它能产生最优编码。

5. (1) 算法描述: 设编码字母集中, 每个字母 $a \in \Sigma$ 的频率是 $f(a)$, 则每次从所有字母中, 选择 $f(a)$ 最小的三个字母, 合并为一棵树, 新树的频率为合并前三字母 (树) 的频率之和。将新树插入优先队列中, 不断抽取频率最小的三棵树合并, 直至剩下一棵树, 对这棵三叉树进行编码, 即为三进制赫夫曼编码。

(2) 需证明其产生最优编码, 须证其贪心选择性质和最优子结构性质。

① 贪心选择性质:

设三叉树 T 表示任一最优前缀码, 下述证一系列操作后, 树 T'' 使 $f(a)$ 最小的字母 x, y, z 为最深结点且互为兄弟, 且 T'' 也前表示最优前缀码。



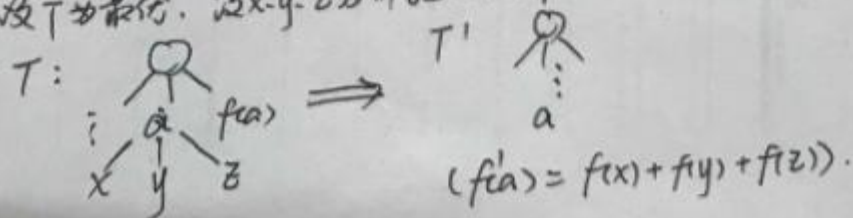
$$\begin{aligned} \because B(T) - B(T') &= f(a)d_T(a) + f(x)d_T(x) - f(a)d_{T'}(a) - f(x)d_{T'}(x) \\ &= (f(a) - f(x)) [d_T(a) - d_T(x)] \geq 0. \end{aligned}$$

则 $B(T) \geq B(T')$. 同理可得: $B(T) \geq B(T') \geq B(T'') \geq B(T''')$

又 T 最优, 则 $B(T) \leq B(T''')$. $\therefore B(T) = B(T''')$ 即 T''' 也最优。

② 最优子结构性质:

设 T 为最优, 设 x, y, z 为 T 的兄弟, a 为父结点, 证如下树 T' :



下要证: T' 是 T 的最优子结构。

$$\because d_T(x) = d_T(y) = d_T(z) = d_{T'}(a) + 1.$$

$$\begin{aligned} \therefore f(x)d_T(x) + f(y)d_T(y) + f(z)d_T(z) &\leq (f(x) + f(y) + f(z)) [d_{T'}(a) + 1] \\ &= f(x) + f(y) + f(z) + f(a)d_{T'}(a) \end{aligned}$$

$$\text{即: } B(T) = B(T') + f(x) + f(y) + f(z).$$

若 T' 不是最优子结构, 则存在 T'' 使 $B(T'') < B(T')$. 将 x, y, z 作为结点加入 T'' 中作为 $a \in \Sigma$, 则得到 T''' .

$$\text{则有 } B(T''') = B(T'') + f(x) + f(y) + f(z).$$

$$< B(T') + f(x) + f(y) + f(z) = B(T).$$

则 T 不是最优, 与假设矛盾, 故 T' 是最优。

(3) 由贪心选择性质和最优子结构性质可得: 上述三进制赫夫曼算法是正确的。
(助教老师, 这是字这么多为什么才20分啊!!!!!!)

辛苦了!!!!

Answer from 詹翁怡

第十周作业

1.设计算法求解N皇后问题，要求给出测试用例，并给出你的程序运行该测试案例之后得到的结果。

N皇后问题研究的是如何将 N个皇后放置在 $N \times N$ 的棋盘上，并且使皇后彼此之间不能相互攻击。

```
vector<string> generateBoard(vector<int> &queens, int n) {
    auto board = vector<string>();
    for (int i = 0; i < n; i++) {
        string row = string(n, '.');
        row[queens[i]] = 'Q';
        board.push_back(row);
    }
    return board;
}

void backtrack(vector<vector<string>> &solutions, vector<int> &queens, int n,
               int row, unordered_set<int> &columns,
               unordered_set<int> &diagonals1, unordered_set<int> &diagonals2) {
    if (row == n) {
        vector<string> board = generateBoard(queens, n);
        solutions.push_back(board);
    } else {
        for (int i = 0; i < n; i++) {
            if (columns.find(i) != columns.end()) {
                continue;
            }
            if (diagonals1.find(row + i) != diagonals1.end()) {
                continue;
            }
            if (diagonals2.find(row - i) != diagonals2.end()) {
                continue;
            }
            queens[row] = i;
            columns.insert(i);
            diagonals1.insert(row + i);
            diagonals2.insert(row - i);
            backtrack(solutions, queens, n, row + 1, columns, diagonals1, diagonals2);
            queens[row] = -1;
            columns.erase(i);
            diagonals1.erase(row + i);
            diagonals2.erase(row - i);
        }
    }
}
```

```

    }
    int diagonal1 = row - i;
    if (diagonals1.find(diagonal1) != diagonals1.end()) {
        continue;
    }
    int diagonal2 = row + i;
    if (diagonals2.find(diagonal2) != diagonals2.end()) {
        continue;
    }
    queens[row] = i;
    columns.insert(i);
    diagonals1.insert(diagonal1);
    diagonals2.insert(diagonal2);
    backtrack(solutions, queens, n, row + 1, columns, diagonals1, diagonals2);
    queens[row] = -1;
    columns.erase(i);
    diagonals1.erase(diagonal1);
    diagonals2.erase(diagonal2);
}
}

vector<vector<string>> solveNQueens(int n) {
    auto solutions = vector<vector<string>>();
    auto queens = vector<int>(n, -1);
    auto columns = unordered_set<int>();
    auto diagonals1 = unordered_set<int>();
    auto diagonals2 = unordered_set<int>();
    backtrack(solutions, queens, n, 0, columns, diagonals1, diagonals2);
    return solutions;
}

```

题目要求给测试案例啊

2.上题如果只让求解N皇后问题不同解法的数目，又该如何设计算法？

将 backtrack 中 `if(row==n){}` 中的生成解法替换为统计解法数目即可

3.设计一个回溯算法求解马周游问题：给出一个8*8的棋盘，一个放在棋盘某个位置上的马是否可以恰好访问每个方格一次，并回到起始位置上？

首先将起点作为当前位置，按照象棋马的移动规则，搜索有没有可以移动的相邻位置；如果有可以移动的相邻位置，则移动到其中的一个相邻位置，并将这个相邻位置作为新的当前位置，按同样的方法继续搜索通往终点的路径；如果搜索不成功，则换另外一个相邻位置，并以它作为新的当前位置继续搜索通往终点的路径。马走日字，当马一开始在黑点 (x,y) 时，它下一步可以到达的点有八个，分别是方向 1: $(x+1,y+2)$ ，方向 2: $(x+2,y+1)$ ，方向 3: $(x+2,y-1)$ ， $\cdots$ ，如图所示。引进增量数组， $dx[] = (1, 2, 2, \cdots)$ ， $dy[] = (2, 1, -1, \cdots)$ ，现在只需要知道方向 k ，下一步的位置就是 $(x+dx[k], y+dy[k])$ 。

令 $flag[0..8;0..8]$ 表示棋盘，并初始化为 0，表示这些位置均未跳过。令 (x_0,y_0) 为起始位置，当前位置点为 (x,y) 。 $route$ 数组纪录访问路线。

JumpHorse(i)

```
1  for k ← 1 to 8 do
2      if  $x + dx[k] \geq 0$  and  $x + dx[k] < 8$  and  $y + dy[k] \geq 0$  and  $y + dy[k] < 8$  then
3          route[i] ← k
4           $x \leftarrow x + dx[k]$ ;  $y \leftarrow y + dy[k]$ 
5          if  $x=x_0$  and  $y=y_0$  then
6              output(i); return
7          else
8              if flag[x, y]=0 then
9                  flag[x, y] ← 1
10                 JumpHorse(i+1)
11              $x \leftarrow x - dx[k]$ ;  $y \leftarrow y - dy[k]$ 
```

4.世界名画陈列馆问题。世界名画陈列馆由 $m \times n$ 个陈列室组成。为了防止名画被盗，需在陈列室设置警卫机器人哨位。每个警卫机器人除了监视它所在的陈列室外，还可以监视与它所在陈列室相邻的上、下、左、右4个陈列室。试设计一个安排警卫机器人哨位的算法，使得名画陈列馆重每一陈列室都在警卫机器人监视之下，且所用的警卫机器人数量最少。

算法思路：按照**从上到下，从左到右**的顺序遍历陈列室，对于按照该顺序遍历到的第一个未被监视的陈列室(设其坐标为 (i, j))，为了使陈列室 $[i,j]$ 受到监视，存在着三种可能是最优的放法：① (i, j) ② $(i+1, j)$ ③ $(i, j+1)$ (因为我们是按照从上到下从左到右的顺序遍历的)。我们可以使用回溯法依次实现这三种放法，并更改陈列室的监视情况，然后搜寻下一个未被监视的陈列室，（实现每种放法的时候记得清除上一种放法对陈列室监视情况的改变）由此使得所有的陈列室进入监视状态,若机器人数量小于最优值则更新。

剪枝优化

- 1.初始机器人数的最优值设置为 $n \times m / 3 + 2$ ，若当前放置的机器人个数大于最优值，则剪枝
- 2.(当前最优值减去已放置机器人个数) $\times 5 <$ 小于未被监视的陈列室,则剪枝


```

const int M = 32, go[5][2]={0,0},{0,1},{0,-1},{1,0},{-1,0}};

int n,m;
int anx[M][M], ans;
int put[M][M], spy[M][M], tmp, spys = 0;
int lim1, lim2;

void search(int i,int j);
void puta(int x,int y,int i,int j)//在(x,y)陈列室放置一个机器人
{
    put[x][y] = 1;
    tmp++;
    for(int i=0; i<5; ++i)
        if(++spy[x + go[i][0]][y + go[i][1]]==1) spys++;

    search(i,j+1);

    put[x][y] = 0;
    tmp--;
    for(int i = 0; i < 5; ++i)
        if(--spy[x + go[i][0]][y + go[i][1]]==0) spys--;
}
void search(int i,int j)//访问 (i,j) 陈列室
{
    if(tmp>=ans) return;//剪枝1
    while(i<=n && spy[i][j]) //已放置的不再被搜索
    {
        ++j;
        if(j>m) ++i, j=1;
    }
    if(i>n) //更新答案
    {
        ans = tmp;
        memcpy(anx, put, sizeof(put));
        return;
    }

    int reach = spys + (ans-tmp)*5;
    if(reach <= lim) return;//剪枝2
    if((i==n&&j==m) || spy[i][j+1]==0) puta(i,j,i,j);
    if(i<n) puta(i+1,j,i,j);
    if(j<m && (spy[i][j+1]==0 || spy[i][j+2]==0)) puta(i,j+1,i,j);
}

```

```

int main()
{
    ans = n * m / 3 + 2;
    lim = n * m;
    for(int i=0;i<=n+1;++i)
        spy[i][0] = spy[i][m+1] = 1;
    for(int i=0;i<=m+1;++i)
        spy[0][i] = spy[n+1][i] = 1;
    search(1,1);
    printf("%d\n",ans );
    for(int i=1;i<=n;++i)
        for(int j=1;j<=m;++j)
            printf("%d%c",anx[i][j],j==m?'\\n':' ');
    return 0;
}

```

5.世界名画陈列馆问题。在上题中，如果要求每一个陈列室仅受一个警卫机器人监视，则应如何设置警卫机器人的哨位？

与题 4 类似，若某点周边已经被监视了，就不应该在该点放置机器人了

算法思路：按照从上到下，从左到右的顺序遍历陈列室，对于未被监视的陈列室 (i,j) ，枚举使其能被监视到的机器人的放置位置，且满足该机器人监视到的区域不与之前已放置的机器人的监视区域重叠。